

CSCE 627 THEORY OF COMPUTABILITY · MODULE 01 · LECTURE NOTES

What a Computation Is

Why the question needed answering.

MODULE OUTCOMES

- Explain why a formal definition of algorithm was necessary.
- State the Church–Turing thesis and classify what kind of claim it is.
- Explain why multiple independent definitions coinciding is the evidence.
- Distinguish a problem from an instance and from a language.
- Explain what an impossibility result is and is not.

1 The question, and the vocabulary

YOU CANNOT PROVE NO ALGORITHM EXISTS UNTIL YOU DEFINE ALGORITHM

Hilbert's tenth problem (1900) asked for a procedure to decide whether a given polynomial equation with integer coefficients has an integer solution. Everyone understood what was being asked, and work on it proceeded for decades.

And nobody could prove that no such procedure exists, because 'procedure' had no definition. You cannot prove a statement quantified over a class you have not specified — which is an obvious point in retrospect and was not obvious at the time.

So the 1930s produced definitions, several at once: Church's lambda calculus, Turing's machines, Gödel and Herbrand's general recursive functions, Post's rewriting systems — all within about two years, by people pursuing different questions.

And they all turned out to define exactly the same class of functions. Which was not expected, and is the central evidence discussed in §2. Hilbert's tenth was then settled negatively in 1970 (Matiyasevich, building on Davis, Putnam and Robinson) — **seventy years after it was asked and thirty-four after it became possible to answer it**, which is a fair measure of how much the definition was worth.

Term	What it means	Example
Instance	One specific input.	'Is 91 prime?' — a question with one answer.
Problem	The whole family of instances.	'Given n , is n prime?' — which is what an algorithm must answer for all n .
Decision problem	A problem whose answer is yes or no.	Primality is a decision problem; factoring is not , though it has a decision version.

Term	What it means	Example
Language	The set of strings whose answer is yes.	$\{ n : n \text{ is prime} \}$, suitably encoded as strings.
Encoding	How objects are represented as strings.	Usually immaterial — but see the callout.

Decision problems are the whole subject, because a yes/no question is a set — the set of inputs whose answer is yes — **and sets of strings are what the mathematics handles well. Most questions can be recast as decision problems without losing anything essential:** instead of 'what is the shortest path', ask 'is there a path of length at most k ', and binary-search on k . **That reduction is why the restriction costs nothing**, and the same move is standard in CSCE 637.

ENCODING USUALLY DOES NOT MATTER, AND KNOWING WHEN IT DOES MATTERS

Any two reasonable encodings of a graph, a program, or a number can be converted into one another in polynomial time — an adjacency matrix to an adjacency list, one source syntax to another — **so decidability is encoding-independent, and so is polynomial-time complexity.** This is why nobody specifies the encoding.

The exception is unary versus binary notation for numbers. A number n written in unary occupies n symbols and in binary occupies about $\log n$, so **the two encodings differ exponentially in length** — which is not a polynomial conversion in one direction.

So an algorithm that is polynomial in the unary input length is exponential in the binary length, and that is exactly what 'pseudo-polynomial' means — CSCE 629's dynamic-programming knapsack algorithm is polynomial in the capacity and exponential in the number of bits of the capacity.

Decidability is entirely unaffected by this and complexity is not — which is one of the clearest differences between this course and CSCE 637, and is worth filing now because the two courses will otherwise blur together.

2 The Church–Turing thesis

THE CHURCH–TURING THESIS IS NOT A THEOREM

The claim is that the intuitive notion of 'effectively computable by a mechanical procedure' coincides exactly with Turing computability.

It cannot be proved, because one side of the claimed equality is informal. It is a claim about the adequacy of a definition — that the formal notion captures the intuitive one — rather than a mathematical statement, and conflating the two is the commonest confusion about it.

The evidence is the coincidence of independent formalisations (Module 11, and the table below): lambda calculus, μ -recursive functions, Post systems, register machines, cellular automata, and every general-purpose programming language ever designed. Each was defined for its own reasons and each defines the same class.

And nothing has ever been proposed that is effectively computable and not Turing computable — ninety years of attempts, including quantum computation, which changes the *speed* of some computations and not the class of computable functions at all. A quantum computer cannot decide the halting problem (CSCE 640 returns to this).

Model	Proposed by	Its nature
Lambda calculus	Church, 1936	Function abstraction and application, with nothing else — no numbers, no data structures, no state.
Turing machines	Turing, 1936	An infinite tape and a finite control, designed explicitly to model a person computing with paper.
μ -recursive functions	Gödel and Herbrand, early 1930s	Composition, primitive recursion, and unbounded minimisation (Module 09).
Post systems	Post, 1936	String rewriting rules.
Register machines	Minsky, 1960s	Two counters and increment, decrement, and test suffice — which is a striking minimality result.
Rule 110, the Game of Life	Cook, Conway	Cellular automata with no control flow at all (Module 11).

These were designed for different purposes by people pursuing different questions — Church was interested in the foundations of logic, Turing in what a human calculator does, Gödel in arithmetic definability — and they coincide exactly. Which is why the thesis is believed rather than merely assumed, and why it is reasonable to treat 'computable' as a settled notion for the rest of the course.

3 What an impossibility result means

- **It says: no single algorithm answers every instance correctly. That is the entire claim**, and reading it as more is the source of nearly every misunderstanding of this subject.
- **It does not say that any particular instance is hard. Most programs you can actually write are obviously terminating or obviously not** — the difficulty lives entirely in the universal quantifier, and a tool that handles the programs people write can be extremely useful (Module 12).
- **It does not forbid a useful tool. Answer 'yes', 'no', or 'I cannot tell', and the resulting problem is decidable** — the third answer is what buys the decidability, and every real analyser uses it.
- **It does not forbid solving a restricted version.** Termination of for loops with constant bounds, recursion on a well-founded order, total functional languages like Agda — **all decidable, by construction**, and **restriction is the other engineering response** (Module 12 §2).
- **And it is permanent. No faster hardware, no new computational model, and no amount of cleverness changes it** — **which is unlike every other result in this program**, where an algorithm can be improved or a model retrained. **'Undecidable' means the problem has no total algorithm, and conflating that with 'intractable in practice' is the mistake to avoid for the next twelve modules.**

WHERE THIS COURSE GOES

Modules 02 and 03: what weaker machines cannot do. Finite memory, and then finite memory plus a stack — **and the proofs there are the course's two techniques in miniature**, on objects simple enough to see all the way through.

Modules 04 through 07: the Turing machine, undecidability, and the structure above it. **Diagonalisation (Module 05) and reduction (Module 06) are the course's two methods**, and between them they establish almost everything else.

Modules 08 through 10: undecidability in real tools, the theory of computable functions, and incompleteness. Gödel's first incompleteness theorem is essentially a corollary of Module 05, which is the most satisfying connection in the course and the reason Module 10 comes after rather than before.

Modules 11 through 13: alternative models, the engineering response to undecidability, and an honest statement of what the limits do and do not constrain.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Sipser — Introduction to the Theory of Computation, chapter 0 and section 3.3	The vocabulary of §1 and the thesis discussion of §2 , stated carefully.

Resource	What it covers for this module
MIT 18.404J — lecture 1 (free video and notes)	Sipser lecturing on this material , which is the best available introduction and is free.
Turing — On Computable Numbers (1936) (free)	The original . Section 9's argument that the machine captures what a human computer does is the thesis's actual justification, and it is readable.
Matiyasevich — Hilbert's Tenth Problem (free chapters)	§1's motivating problem, resolved — and a good illustration of how long a reduction can take to construct.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write three decision problems from your own work** and express each as a language.
- 2 **Find a problem that is not naturally a decision problem** and recast it as one. State what was lost.
- 3 **Show that two encodings of a graph convert in polynomial time.**
- 4 **Show that unary and binary do not**, and relate it to CSCE 629's knapsack algorithm.
- 5 **Write down, in your own words, what kind of claim the Church–Turing thesis is**, and why it cannot be proved.
- 6 **Find a claimed counterexample to the thesis** and identify what it assumes (usually infinite precision or infinite time).
- 7 **For three programs you have written, decide whether they terminate** and note how long each took to settle.
- 8 **Write a program whose termination you cannot determine.**
- 9 **Find a tool that answers 'I cannot tell'** and note how often it does.
- 10 **Write down what you expect to be impossible**, and keep it for Module 13.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why could Hilbert's tenth problem not be answered in 1900?
- 2 Define instance, problem, decision problem, and language.
- 3 Why is a decision problem a set?
- 4 When does encoding matter, and what is the term for the consequence?
- 5 What kind of claim is the Church–Turing thesis?

- 6 What is the evidence for it, and why is independence important?
- 7 Does quantum computation affect it?
- 8 State four things 'undecidable' does not mean.
- 9 What makes this course's results different from the rest of the program's?